

46-927, Spring 2021: Homework #2

Due **1:30pm** Eastern, February 10, 2021.

Homework should be submitted electronically on Gradescope before the deadline. Please submit your homework as a single PDF file. We recommend using Jupyter notebooks to prepare your homework.

Do all computational problems in Python unless otherwise noted. Please post any questions on the Piazza discussion board.

1. *Gradient boosting on the marketing data set. (The end of the marketing data set?)*
We're going to apply XGBoost to the marketing data set (from homework 5 and homework 6 in ML1) to see if we can beat the random forest.

In the previous problem, we only had numeric covariates. In this data set, we also have categorical variables. The `xgboost` library can only handle numeric matrices and vectors for X and y , so you need to deal with dummy coding. You also need to convert to the `xgboost DMatrix` format as discussed in class.

Load the marketing data set, like last term. Transform the data to dummy code your categorical variables, and encode your outcome variable as $\{0,1\}$. Doing this on the whole data set at once is easier than trying to make it match on every split.

Now split the data into a training set, a test set, and final test set, in proportions 60%, 20%, 20%. Do this using the following code, keeping the seeds the same so that we can compare.

```
from sklearn.model_selection import train_test_split
X_tmp, X_final, y_tmp, y_final = train_test_split(X, y,
                                                    test_size=0.20,
                                                    random_state=1)
X_train, X_test, y_train, y_test = train_test_split(X_tmp, y_tmp,
                                                    test_size=0.25,
                                                    random_state=10)
```

IMPORTANT: You should be using `X_train`, `y_train` for all the fitting. You should be using `X_test`, `y_test` for all the evaluation of models in parts (a)-(d). You should not touch `X_final`, `y_final` until part (e). Don't cheat and peek at performance on this data set, you'll ruin a potential surprise.

For the version you submit, please turn off all the output that the CV and training functions spit out by setting the `verbosity` parameter to 0.

- (a) First, fit a simple boosted tree model to the data set, using default parameters where possible:

```
{  
  'learning_rate': 0.3,  
  'max_depth': 6,  
}
```

in the `xgb.train` function. Set the `objective` to `binary:logistic`, and fix a `seed`. Set the `eval_metric` to `auc` for when we start tuning. We'll start with AUC because we know misclassification rate is a mess on this data, and it will give us pretty ROC curves to compare. Tune the number of trees using `xgb.cv`.

Fit this default model and use the `predict` function to obtain a classification score. Make predictions by thresholding at 0.5. Compare misclassification error, AUC, and best 1000 to the random forest from Homework 6 in ML1, and make an ROC plot with both models.

(Remember the “best 1000” is the accuracy if you take the 1000 clients in the test set who are predicted by your model to be the most likely to have a successful sale (`y = yes`). Your best 1000 accuracy is the fraction of those clients whose real label is “yes”.)

- (b) Try tuning the model yourself. All parameters are fair game, as are `sklearn` cross-validation functions or hand tuning. Just explain your reasoning. **Your goal here is to improve your best 1000 set performance.**

Note: There is no objective or evaluation metric in `xgboost` that directly measures your best 1000 set performance, so you'll have to optimize other things as proxies.

There's no one right answer to this problem. Take a look at the available parameters and try them out.

Try playing with any parameters you wish. I'd suggest at least looking at least at `max_depth`, `scale_pos_weight`, `min_child_weight`, `subsample`, `gamma`, etc. Be careful about brute force cross-validating all of the parameters at once, both because of a risk of overfitting and massive computational time. You'll have to be clever. Remember to adjust your number of trees from time to time.

Never touch your final held out set during any of this tuning, so that we have a final chance to check everything fairly.

What is the best set of model parameters that you can find? For your best model, produce the same metrics as in part (a), and make a new ROC plot with all three models.

It's easy to accidentally sink lots of time into tuning. For the purpose of this homework, it's not really worth spending more than 30 minutes here actually experimenting, unless you're really having fun. Just stop at that point.

Hints:

You can find a list of parameters here:

<http://xgboost.readthedocs.io/en/latest/parameter.html>

You can find some reasonable tuning strategies here:

https://xgboost.readthedocs.io/en/latest/tutorials/param_tuning.html

and here:

<https://www.analyticsvidhya.com/blog/2016/03/complete-guide-parameter-tuning-xgb/>

- (c) **Finally, now that you're all done tuning.** For the best model from the previous part, find the predictions for the final held-out set. You should not have touched this set until now. Also make predictions on this set with the random forest.

Plot the ROC curves together, and report the misclassification rates, AUC, and best 1000 performance. How do they compare to the performance you saw in the last part? Did your improvements survive the change to the new data set?

However this turns out, don't go back and change anything about your tuning, the comparison is the interesting part.

After all your tuning effort, were you able to beat the random forest?

DO THIS: For potential use later, please submit the performance of your fully tuned model on both test data sets in the following Google form:

<https://forms.gle/96dmSV1Jbw61K67QA>

This won't impact your grade in any way (and is anonymous), but will produce something potentially interesting.

2. Classifying fraudulent banknotes. We will consider a small data set of valid and fraudulently printed banknotes. Because of the small data set, we'll be considering a simple model: LDA.

Load the data from `banknote_measurements.csv`. The first column indicates whether a banknote was fraudulent; code these as 1 and genuine bills as 0. The other 5 columns are measurements of the printed banknotes in millimeters. We would expect fraudulent bills to potentially deviate from the usual distribution of genuine bill dimensions.

Make a train-test split with:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5,
                                                    random_state=1)
```

- (a) Plot overlapping histograms of each of the five feature, colored by bill status. That is, make five plots, each showing two histograms overlapped in different colors. You may use the following coding approach to overlap two semi-transparent histograms

```
ax.hist(X_test[column_name][y_test==0], alpha=0.5)
ax.hist(X_test[column_name][y_test==1], alpha=0.5)
```

Do any of the features perfectly separate the two classes?

- (b) Now fit LDA to classify the bills. You may use `predict()` to classify (equivalent to thresholding probabilities at 0.5) since the classes are balanced in this data set. Generate a confusion matrix using `confusion_matrix`. What is your misclassification rate?
- (c) Let's see why we do so well. Transform your `X_test` data based on your LDA fit. To do this, use the `fit.transform` function of the returned LDA object. Because we only have two classes, the relevant "PCA-like" space is only 1-dimensional. This is what the `transform` function returns. Make another plot with a pair of histograms of these scores overlapped, colored by the bill type. This is your transformed space. Compare how well this plot separates your bill types to your previous univariate plots.

3. *Implementing your own Naive Bayes.* One nice thing about Naive Bayes is its flexibility. You can model each variable (conditional on class) with a separate univariate distribution. However, machine learning packages usually only implement a few versions of Naive Bayes, limiting your choices. `sklearn` implements Gaussian, Multinomial, and Bernoulli. In this problem, you'll build your own simple Naive Bayes classifier on a small data set, using a different model for each variable. To do this problem, don't directly use any Naive Bayes packaged functions from python.

We'll use the simple default data set we've plotted a few times in class. You can download `default_train.csv` and `default_test.csv` from Canvas. Fit models below on the training data and evaluate them on the test data.

- (a) You will be predicting default based on credit balance, income, and whether or not an individual is a student. The first two are continuous, and we'll just model them as Gaussian within each class for simplicity. The last is binary, which we'll model as Bernoulli within class.

Plot histograms of each of the variables (using the training data).

Also, to see if you believe the Naive Bayes model – which pretends that all variables are independent conditional on the value of default – for each pair of predictor variables, make side-by-side plots such that:

- One plot corresponds to all data points with `default==1` and the other corresponds to all data points with `default==0`.
- The two predictor variables are plotted against one another: scatter plots if looking at two continuous variables, overlaid histograms if looking at a continuous and a discrete variable.

Based on these plots, do you see anything that makes you doubt the Naive Bayes model?

(Whatever you decide... we're still going to fit one.)

- (b) To build a Naive Bayes model, you need to fit a distribution to every predictor conditioned on each state of default. For example:
- For balance, fit a Gaussian to balance for everyone who defaulted, and a separate Gaussian to balance for everyone who did not default. We will call these $f(\text{balance}|\text{default})$ and $f(\text{balance}|\text{no default})$.
 - For income, fit a Gaussian to income for everyone who defaulted, and a separate Gaussian to income for everyone who did not default. We will call these $f(\text{income}|\text{default})$ and $f(\text{income}|\text{no default})$.
 - For student, fit a Bernoulli to student for everyone who defaulted, and a separate Bernoulli to student for everyone who did not default. You will get a $P(\text{student}|\text{default})$ and $P(\text{student}|\text{no default})$.

Use the usual maximum likelihood estimates for the parameters of each distribution. These estimates should be formed using the training data. Report the parameters of your distributions.

You will also need an estimate of the prior probability of default. Form a simple estimate, based on your training data, of $P(Y = 1)$

- (c) Finally, we get to classify! Remember that we want to classify to the group with the largest $P(X|Y = k)P(Y = k)$. Because we'll be multiplying a bunch of

densities and probabilities, we'll find it easier to work with the log. To classify each point, you will compare

$$\begin{aligned} &\log P(\text{default}) + \log f(\text{balance}|\text{default}) + \log f(\text{income}|\text{default}) \\ &\quad + \log P(\text{student}|\text{default}) \end{aligned}$$

to

$$\begin{aligned} &\log P(\text{no default}) + \log f(\text{balance}|\text{no default}) + \log f(\text{income}|\text{no default}) \\ &\quad + \log P(\text{student}|\text{no default}) \end{aligned}$$

Carry out these classifications on your test set.

Produce:

- A confusion matrix of your results, if you just classify to the {default, no default} with the highest value.
- A misclassification rate corresponding to the same classifier.
- An ROC curve for the classifier. To do this, you need a score, rather than a binary classification. You can use:

$$\begin{aligned} &(\log P(\text{default}) + \log f(\text{balance}|\text{default}) + \log f(\text{income}|\text{default}) + \log P(\text{student}|\text{default})) \\ &- (\log P(\text{no default}) + \log f(\text{balance}|\text{no default}) + \log f(\text{income}|\text{no default}) \\ &\quad + \log P(\text{student}|\text{no default})) \end{aligned}$$

Feel free to use the built in ROC function here.

- (d) Suppose that you were not happy about using a Gaussian to model your continuous variables. After all, look at your plot of income! Rebuild your model, but this time use kernel density estimation to estimate the distributions of income and balance conditional on default status. You can use `KernelDensity` from `sklearn.neighbors`, which can produce log density values, or other functions if you prefer. Feel free to pick a reasonable bandwidth by just looking at the fit on your training data (or you may cross-validate if you wish). Using this model, produce the same outputs that you did in part (c).